COER UNIVERSITY

DBMS LAB ASSIGNMENT | LAB SHEET 5: SQL JOINS

Lab Sheet 5: SQL Joins

Question 1

Question: Demonstrate an INNER JOIN between Student and Department.

Answer:

```
SELECT * FROM Student S INNER JOIN Department D ON S.DeptID = D.DeptID;
```

Explanation: INNER JOIN returns rows where key values (DeptID) match in both tables.

Question 2

Question: Display student names along with their department names using INNER JOIN.

Answer:

```
SELECT S.Name AS StudentName, D.DeptName FROM Student S INNER JOIN Department D ON
S.DeptID = D.DeptID;
```

Explanation: Projects student name and matching department name.

Question 3

Question: Display employee names along with their department names.

Answer:

```
SELECT E.Name AS EmployeeName, D.DeptName FROM Employee E INNER JOIN Department D ON
E.DeptID = D.DeptID;
```

Explanation: Joins Employee and Department tables on DeptID.

Question 4

Question: Display students along with their course details.

Answer:

```
SELECT S.Name AS StudentName, C.CourseName, C.Fees FROM Student S INNER JOIN Course C
ON S.CourseID = C.CourseID;
```

Explanation: Joins Student and Course tables on CourseID.

Question 5

Question: Find students belonging to a particular department using INNER JOIN.

Answer:

```
SELECT S.Name, D.DeptName FROM Student S INNER JOIN Department D ON S.DeptID = D.DeptID
WHERE D.DeptName = 'Computer Science';
```

Explanation: Filters inner join results using a WHERE condition on department name.

Question 6

Question: Demonstrate a LEFT OUTER JOIN between Student and Department.

Answer:

```
SELECT S.Name, D.DeptName FROM Student S LEFT JOIN Department D ON S.DeptID = D.DeptID;
```

Explanation: LEFT JOIN returns all rows from the left table (Student), filling missing department data with NULL.

Question 7

Question: Display all students, including those who do not have a matching department.

Answer:

```
SELECT S.Student_ID, S.Name, D.DeptName FROM Student S LEFT JOIN Department D ON
S.DeptID = D.DeptID;
```

Explanation: LEFT JOIN retains students without an assigned department.

Question 8

Question: Demonstrate a RIGHT OUTER JOIN between Student and Department.

Answer:

```
SELECT S.Name, D.DeptName FROM Student S RIGHT JOIN Department D ON S.DeptID =
D.DeptID;
```

Explanation: RIGHT JOIN returns all rows from the right table (Department), filling missing student data with NULL.

Question 9

Question: Display all departments, including departments having no students.

Answer:

```
SELECT D.DeptID, D.DeptName, S.Name AS StudentName FROM Student S RIGHT JOIN Department
D ON S.DeptID = D.DeptID;
```

Explanation: Lists all departments, displaying NULL for departments without enrolled students.

Question 10

Question: Find students who are not assigned to any department using LEFT JOIN.

Answer:

```
SELECT S.Student_ID, S.Name FROM Student S LEFT JOIN Department D ON S.DeptID =
D.DeptID WHERE S.DeptID IS NULL;
```

Explanation: LEFT JOIN combined with WHERE S.DeptID IS NULL identifies unassigned students.

Question 11

Question: Find departments having no students using RIGHT/LEFT JOIN.

Answer:

```
SELECT D.DeptID, D.DeptName FROM Department D LEFT JOIN Student S ON D.DeptID =
S.DeptID WHERE S.Student_ID IS NULL;
```

Explanation: LEFT JOIN with WHERE S.Student_ID IS NULL filters for empty departments.

Question 12

Question: Demonstrate a FULL OUTER JOIN concept using suitable queries/workaround.

Answer:

```
SELECT S.Name AS StudentName, D.DeptName FROM Student S LEFT JOIN Department D ON
S.DeptID = D.DeptID
UNION
SELECT S.Name AS StudentName, D.DeptName FROM Student S RIGHT JOIN Department D ON
S.DeptID = D.DeptID;
```

Explanation: Emulates FULL OUTER JOIN in MySQL by executing a UNION of LEFT JOIN and RIGHT JOIN.

Question 13

Question: Demonstrate a CROSS JOIN between Student and Course.

Answer:

```
SELECT S.Name, C.CourseName FROM Student S CROSS JOIN Course C;
```

Explanation: Produces a Cartesian product (m x n rows) pairing every student with every course.

Question 14

Question: Calculate the number of possible Student-Course combinations using CROSS JOIN.

Answer:

```
SELECT COUNT(*) AS TotalCombinations FROM Student S CROSS JOIN Course C;
```

Explanation: Counts total Cartesian product rows.

Question 15

Question: Demonstrate a NATURAL JOIN between two suitable tables.

Answer:

```
SELECT * FROM Student NATURAL JOIN Department;
```

Explanation: NATURAL JOIN automatically matches columns that share identical names (DeptID) in both tables.

Question 16

Question: Compare the result of NATURAL JOIN and INNER JOIN.

Answer:

```
-- 1. NATURAL JOIN (implicit column match)
SELECT * FROM Student NATURAL JOIN Department;

-- 2. INNER JOIN (explicit key definition)
SELECT * FROM Student S INNER JOIN Department D ON S.DeptID = D.DeptID;
```

Explanation: NATURAL JOIN automatically matches common column names (DeptID) and removes duplicate key columns, whereas INNER JOIN requires an explicit ON clause.

Question 17

Question: Perform a SELF JOIN on an Employee table to show employee-manager relationships.

Answer:

```
SELECT E1.Name AS Employee, E2.Name AS Manager FROM Employee E1 LEFT JOIN Employee E2
ON E1.ManagerID = E2.EmpID;
```

Explanation: Joins the Employee table with itself to map each employee's ManagerID to their manager's EmpID.

Question 18

Question: Display employees along with their managers using SELF JOIN.

Answer:

```
SELECT E1.EmpID, E1.Name AS EmployeeName, E2.Name AS ManagerName FROM Employee E1 INNER
JOIN Employee E2 ON E1.ManagerID = E2.EmpID;
```

Explanation: Projects employee ID, employee name, and assigned manager name.

Question 19

Question: Join three tables—Student, Department and Course.

Answer:

```
SELECT S.Name, D.DeptName, C.CourseName FROM Student S INNER JOIN Department D ON
S.DeptID = D.DeptID INNER JOIN Course C ON S.CourseID = C.CourseID;
```

Explanation: Chains two INNER JOIN clauses across three tables.

Question 20

Question: Display student name, course name and department name using multiple joins.

Answer:

```
SELECT S.Name AS StudentName, C.CourseName, D.DeptName FROM Student S JOIN Course C ON S.CourseID = C.CourseID JOIN Department D ON S.DeptID = D.DeptID;
```

Explanation: Retrieves consolidated multi-table records.

Question 21

Question: Find employees whose salary is greater than the average salary of their department using joins/subqueries.

Answer:

```
SELECT E.Name, E.Salary, E.DeptID FROM Employee E WHERE E.Salary > (SELECT AVG(Salary) FROM Employee WHERE DeptID = E.DeptID);
```

Explanation: Uses a correlated subquery to filter employees earning above their department average.

Question 23

Question: Display departments and the total number of students in each department.

Answer:

```
SELECT D.DeptName, COUNT(S.Student_ID) AS TotalStudents FROM Department D LEFT JOIN Student S ON D.DeptID = S.DeptID GROUP BY D.DeptID, D.DeptName;
```

Explanation: Combines LEFT JOIN, GROUP BY, and COUNT().

Question 24

Question: Display courses along with the number of students enrolled in each course.

Answer:

```
SELECT C.CourseName, COUNT(S.Student_ID) AS EnrolledStudents FROM Course C LEFT JOIN Student S ON C.CourseID = S.CourseID GROUP BY C.CourseID, C.CourseName;
```

Explanation: Groups by course and counts enrolled students.

Question 25

Question: Create a complete query using INNER JOIN, LEFT JOIN, GROUP BY and aggregate functions to generate a student-department-course report.

Answer:

```
SELECT D.DeptName, C.CourseName, COUNT(S.Student_ID) AS TotalEnrolled, AVG(S.Fees) AS
AvgFeesPaid
FROM Department D
INNER JOIN Student S ON D.DeptID = S.DeptID
LEFT JOIN Course C ON S.CourseID = C.CourseID
GROUP BY D.DeptName, C.CourseName;
```

Explanation: Comprehensive query uniting INNER JOIN, LEFT JOIN, GROUP BY, COUNT(), and AVG().